
CENP6206 – BIG DATA: HIGH- PERFORMANCE COMPUTING

1. OVERVIEW

1.1 Course Objectives (Expanded)

The purpose of this course is to equip students with the theoretical foundations and practical tools required to process, store, analyze, and visualize massive datasets using High-Performance Computing (HPC) technologies.

After completing this course, students should be able to:

1. Explain why traditional computing fails for Big Data workloads.
2. Design scalable distributed algorithms.
3. Use parallel programming models to accelerate computation.
4. Deploy Big Data systems on clusters and cloud environments.
5. Evaluate different storage and processing frameworks.
6. Implement Big Data workflows using Hadoop and Spark ecosystems.

1.2 Why HPC is Essential for Big Data

The Data Explosion

We live in the era of **datafication** — every activity produces data:

Domain	Examples of Data Generated
Social media	Posts, images, videos
IoT	Sensors, smart homes
Healthcare	Genomics, medical imaging
Finance	Transactions, fraud logs
Science	Climate simulations

Global data doubles approximately **every 2 years**.

Traditional computing fails because:

- A single machine cannot store petabytes.
- Sequential processing is too slow.
- Systems must tolerate hardware failures.

Role of HPC

HPC allows:

- Thousands of processors working simultaneously.
- Massive distributed storage systems.
- Near real-time analytics.

Without HPC, Big Data analytics would take months or years.

2 BIG DATA & PARALLEL PROGRAMMING MODELS

2.1 Understanding Big Data Beyond the 5Vs

Traditionally, Big Data has been defined using the **5Vs**: Volume, Velocity, Variety, Veracity and Value.

However, modern Big Data systems go beyond these characteristics. The true challenge of Big Data lies in **complexity, speed, uncertainty and interpretation**.

Big Data is therefore not simply “large data”.

It is **complex, rapidly evolving, distributed, and difficult to process using traditional systems**.

Additional Vs of Big Data

1. Variability

Variability refers to **changes in data meaning over time**.

Unlike traditional datasets that remain static, Big Data is dynamic and context-dependent.

Examples:

- Social media sentiment changes daily.
- Stock market data changes every second.
- User behaviour evolves over time.

Why variability is challenging:

- Models trained today may become obsolete tomorrow.
- Requires real-time analytics and adaptive algorithms.

This leads to the need for:

- Streaming analytics
- Machine learning pipelines

- Continuous model retraining

2. Visualization

Visualization refers to the **ability to interpret and communicate insights from massive datasets**.

Problem:

Humans cannot understand billions of rows of raw data.

Therefore, Big Data must be:

- Aggregated
- Summarized
- Represented visually

Examples of Big Data visualization:

- Real-time dashboards
- Heat maps
- Interactive charts
- Geographic maps

Visualization is crucial because:

Big Data has no value unless insights can be understood.

3. Volatility

Volatility refers to the **lifespan of data**.

Not all Big Data must be stored forever.

Examples:

- Sensor data may only be useful for hours.
- Network logs may be needed for days.
- Financial records may be needed for years.

Organizations must decide:

- How long data should be stored.
- When data should be archived or deleted.

This leads to concepts such as:

- Data lifecycle management
- Tiered storage (hot, warm, cold data)

2.2 Big Data Processing Pipeline

Big Data is processed through a **multi-stage pipeline**.

Stage 1 - Data Generation

Data is produced from many sources:

- IoT devices
- Social media platforms
- Mobile apps
- Scientific experiments
- Financial transactions

Modern systems generate **petabytes daily**.

Stage 2 - Data Ingestion

Data must be collected and transported into storage systems.

Tools used:

- Apache Kafka
- Flume
- Logstash

Key challenges:

- Handling high data velocity.
- Managing real-time data streams.
- Ensuring data reliability.

Stage 3 - Data Storage (HPC Role Begins)

Traditional databases cannot store massive datasets efficiently.

Big Data uses:

- Distributed file systems (HDFS)
- Cloud storage (S3, Blob Storage)

Key concepts:

- Data partitioning
- Replication
- Fault tolerance

Why HPC is needed:

- Data stored across thousands of nodes.

Stage 4 - Data Processing (Major HPC Role)

This is the **core stage** of Big Data.

Tasks include:

- Filtering
- Sorting
- Aggregation
- Machine learning

Processing frameworks:

- Hadoop MapReduce
- Apache Spark

Parallel computing is essential here.

Stage 5 - Analytics (HPC Role)

Advanced analytics include:

- Predictive modeling
- Data mining
- Deep learning

These tasks require:

- Massive computational power
- GPUs and clusters

Stage 6 - Visualization

Final stage:

- Present insights
- Support decision making

Tools:

- Tableau
- Power BI
- Superset

Why HPC is Critical in Steps 3–5

Stage	HPC Role
Storage	Distributed file systems
Processing	Parallel algorithms
Analytics	Machine learning & AI

2.3 Parallel Computing Deep Dive

Big Data processing is impossible without **parallel computing**.

Why Parallelism Improves Performance

Total execution time:

$$T_{total} = T_{compute} + T_{communication}$$

Where:

- $T_{compute}$ = Time spent computing.
- $T_{communication}$ = Time spent transferring data between nodes.

Goal:

- Reduce computation time by splitting tasks.
- Reduce communication overhead.

Example: Sequential vs Parallel Processing

Sequential:

- Process 1TB data on one computer → 10 hours.

Parallel (10 nodes):

- Each node processes 100GB → 1 hour.

Speedup $\approx 10\times$.

Key Benefits of Parallel Computing

- Faster processing
- Ability to handle huge datasets
- Scalability
- Fault tolerance

2.4 Granularity of Parallelism

Granularity describes the **size of tasks** in a parallel system.

Type	Description
Fine-grained	Many small tasks
Coarse-grained	Few large tasks

Fine-Grained Parallelism

Characteristics:

- Frequent communication.
- Small tasks.
- High synchronization cost.

Examples:

- Scientific simulations
- Weather modeling

Coarse-Grained Parallelism

Characteristics:

- Large tasks.
- Minimal communication.
- High scalability.

Examples:

- MapReduce jobs
- Big Data analytics

Big Data systems prefer coarse-grained tasks because communication between nodes is expensive.

2.5 Shared vs Distributed Memory Systems

Shared Memory Systems

All processors share the same memory space.

Examples:

- Multicore CPUs
- Laptops and desktops

Advantages:

- Easy programming.
- Fast communication via shared variables.

Limitations:

- Limited scalability.
- Expensive hardware.
- Memory bottlenecks.

Distributed Memory Systems

Each node has its own memory.

Nodes communicate using **message passing**.

Examples:

- Hadoop clusters
- Cloud clusters

Advantages:

- Extremely scalable.
- Fault tolerant.
- Cost-effective.

Challenges:

- Complex programming.
- Communication overhead.

Deep Comparison Table

Feature	Shared Memory	Distributed Memory
Communication	Shared variables	Message passing
Scalability	Limited	Very high
Cost	Expensive	Cheap commodity hardware
Fault tolerance	Low	High
Programming complexity	Easier	Harder
Example	Multicore CPU	Hadoop Cluster

Why Big Data Uses Distributed Memory

Big Data systems require:

- Thousands of machines.
- Petabyte storage.
- Fault tolerance.

Shared memory cannot scale this far.

Therefore:

Big Data = Distributed Memory Computing.

SUMMARY

Key takeaways:

- Big Data is about complexity and speed, not just size.
- HPC supports storage, processing and analytics.
- Parallel computing reduces execution time.
- Coarse-grained tasks suit Big Data.
- Distributed memory systems enable scalability.

REVISION QUESTIONS

Short Answer Questions

1. Explain the additional Vs of Big Data beyond the traditional 5Vs.
2. Describe the stages of the Big Data processing pipeline.
3. Why is HPC critical in Big Data storage and processing?
4. Differentiate between fine-grained and coarse-grained parallelism.
5. Explain the equation $T_{total} = T_{compute} + T_{communication}$.

Exam Essay Questions

1. Discuss the Big Data processing pipeline and explain the role of HPC in each stage.
2. Compare shared memory and distributed memory systems and justify why distributed systems are preferred in Big Data.
3. Explain how parallel computing improves Big Data processing performance with examples.

3 GRID COMPUTING VS CLOUD COMPUTING

3.1 Introduction

Before cloud computing became dominant, **Grid Computing** was the primary approach used to perform large-scale scientific and research computations.

Both Grid and Cloud computing aim to:

- Provide large computational power
- Enable distributed resource sharing
- Support Big Data processing

However, their **design philosophy, management and usage models differ significantly.**

3.2 What is Grid Computing?

Definition

Grid computing is a **distributed computing infrastructure** that connects multiple computers from different locations and organizations to work together on a common problem.

A grid is essentially a **virtual supercomputer** built from geographically distributed resources.

Key Idea of Grid Computing

Instead of buying one powerful supercomputer, organizations:

- Connect many smaller computers
- Share computational resources
- Solve large scientific problems collaboratively

Characteristics of Grid Computing

- Resources belong to **multiple organizations**
- Highly distributed and geographically separated
- Focus on **scientific and research workloads**
- Uses **batch processing**

- Resource sharing is based on **collaboration**

Typical Grid Computing Applications

Grid computing has historically been used in:

- Climate modeling
- Space exploration
- Physics simulations (CERN experiments)
- Bioinformatics and genome research
- Engineering simulations

Example:

The **Large Hadron Collider (LHC)** uses grid computing to process petabytes of experimental data.

Grid Computing Architecture

Grid systems typically contain:

1. Resource Layer

Physical resources:

- Computers
- Storage systems
- Networks

2. Middleware Layer

Software that enables resource sharing:

- Job scheduling
- Security
- Resource discovery

Examples:

- Globus Toolkit
- gLite

3. Application Layer

Scientific applications run on the grid.

Limitations of Grid Computing

Despite its power, grid computing has several limitations:

- Difficult to manage and configure
- Requires coordination between organizations
- Limited scalability
- No on-demand resource provisioning
- Not user-friendly for businesses

These limitations led to the development of **Cloud Computing**.

3.4 What is Cloud Computing?

Definition

Cloud computing provides **on-demand computing services over the internet**, allowing users to access computing resources without owning physical hardware.

Cloud computing introduced a **service-oriented and commercial model**.

Key Idea of Cloud Computing

Instead of owning infrastructure:

- Rent computing power when needed
- Pay only for what you use
- Scale resources dynamically

This is called the **utility computing model**.

Essential Characteristics of Cloud Computing

According to NIST, cloud computing has five core characteristics:

1. On-Demand Self-Service

Users can provision resources automatically without human interaction.

Example:

Launching a virtual machine in minutes.

2. Broad Network Access

Resources are accessible via the internet from:

- Phones
- Laptops

- Servers

3. Resource Pooling

Cloud providers pool resources and serve multiple customers.

This is called **multi-tenancy**.

4. Rapid Elasticity

Resources can scale up or down instantly.

Example:

A website handling millions of users during peak traffic.

5. Measured Service

Users pay based on usage:

- Storage used
- Compute hours
- Network traffic

3.5 Cloud Service Models

1. Infrastructure as a Service (IaaS)

Provides:

- Virtual machines
- Storage
- Networking

User controls:

- OS
- Applications

Examples:

- AWS EC2
- Microsoft Azure VM

2. Platform as a Service (PaaS)

Provides:

- Development platforms
- Runtime environments

User focuses on:

- Writing code
- Deploying applications

Examples:

- Google App Engine
- Heroku

3. Software as a Service (SaaS)

Provides ready-to-use software.

Examples:

- Gmail
- Office 365
- Dropbox

3.6 Cloud Deployment Models

Model	Description
Public Cloud	Services available to public via internet
Private Cloud	Dedicated infrastructure for one organization
Hybrid Cloud	Combination of public and private cloud

3.7 Grid vs Cloud Computing

This comparison is **very important for exams**.

Feature	Grid Computing	Cloud Computing
Ownership	Multiple organizations	Single provider
Resource access	Collaborative sharing	On-demand rental
Scalability	Limited	Highly elastic
Virtualization	Minimal	Core technology
Cost model	Usually free/research	Pay-as-you-go
Ease of use	Complex	User-friendly
Provisioning	Manual	Automated
Reliability	Moderate	High
Target users	Scientists	Businesses & developers

Key Concept: Virtualization

Cloud computing relies heavily on **virtualization**.

Virtualization allows:

- Multiple virtual machines on one physical machine
- Efficient resource utilization
- Isolation between users

Types:

- Server virtualization
- Storage virtualization
- Network virtualization

3.8 Why Cloud is Better for Big Data

Big Data workloads require:

- Massive storage
- High computational power
- Scalability
- Fault tolerance

Cloud provides all these features.

Advantages of Cloud for Big Data

1. Elastic Scalability

Clusters can grow from:

- 10 nodes → 10,000 nodes in minutes.

2. Cost Efficiency

No need to buy expensive hardware.

3. High Availability

Data replicated across regions.

4. Managed Big Data Services

Examples:

- Hadoop as a Service
- Spark as a Service

3.9 Grid and Cloud: Complementary Technologies

Grid computing still exists in:

- Scientific research
- High-energy physics
- Academic collaborations

Cloud computing dominates:

- Business analytics
- AI/ML
- Web services
- Big Data platforms

SUMMARY

Key takeaways:

- Grid computing focuses on **resource sharing across organizations**.
- Cloud computing focuses on **on-demand service delivery**.
- Cloud uses virtualization and elastic scaling.
- Cloud is the preferred platform for Big Data analytics.

REVISION QUESTIONS

Short Answer

1. Define grid computing.
2. List the five characteristics of cloud computing.
3. Differentiate between IaaS, PaaS and SaaS.
4. Explain cloud deployment models.
5. Why is virtualization important in cloud computing?

Essay Questions

1. Compare and contrast grid and cloud computing with examples.
2. Discuss why cloud computing is better suited for Big Data processing.
3. Explain the cloud service models and their applications.

4 GPGPU FOR BIG DATA APPLICATIONS

4.1 Introduction

As Big Data analytics became more computationally demanding, traditional CPUs were no longer sufficient to handle the enormous workloads efficiently.

This led to the use of **General-Purpose Graphics Processing Units (GPGPU)** for non-graphics computing.

Originally designed for rendering images and video, GPUs are now widely used in:

- Machine learning
- Scientific simulations
- Big Data analytics
- Artificial intelligence
- Deep learning

Today, GPUs are considered a **core component of High-Performance Computing (HPC)**.

4.2 What is GPGPU?

Definition

GPGPU (General-Purpose GPU computing) refers to using the GPU to perform **general computational tasks** instead of graphics processing.

This approach enables **massive parallelism**, which is ideal for Big Data workloads.

4.3 Why CPUs Are Not Enough for Big Data

Traditional CPUs are designed for **sequential processing** and complex logic.

They excel at:

- Running operating systems
- Handling control flow
- Executing diverse tasks

But Big Data workloads involve:

- Processing billions of records
- Performing repeated mathematical operations
- Running machine learning algorithms

These tasks require **massive parallel processing**, which CPUs cannot provide efficiently.

4.4 GPU Architecture Overview

A GPU contains **thousands of small cores** designed for executing many tasks simultaneously.

CPU Architecture

- Few powerful cores (4–32 cores)
- Optimized for latency
- Designed for sequential tasks

GPU Architecture

- Thousands of smaller cores
- Optimized for throughput
- Designed for parallel tasks

4.5 CPU vs GPU Comparison

Feature	CPU	GPU
Number of cores	Few (4–32)	Thousands
Processing type	Sequential	Parallel
Optimization	Low latency	High throughput
Best for	Complex logic	Large-scale computation
Example tasks	OS, databases	ML, Big Data, simulations

Key Insight

CPU = Brain (decision making)

GPU = Factory (mass production)

Big Data analytics requires **factory-style processing**.

4.6 How GPUs Accelerate Big Data

Big Data workloads often involve:

- Matrix operations
- Vector calculations

- Sorting and searching
- Deep learning training

These tasks can be split into thousands of smaller tasks and executed in parallel.

Example:

Training a neural network:

- CPU: weeks
- GPU: hours

4.7 Parallel Processing on GPU

GPUs use the **SIMD model**:

Single Instruction, Multiple Data

Meaning:

One instruction runs on thousands of data points simultaneously.

Example:

Add 1 to 1 billion numbers.

- CPU: one-by-one
- GPU: thousands at once

4.8 CUDA and OpenCL

To program GPUs, special frameworks are used.

CUDA (Compute Unified Device Architecture)

Developed by NVIDIA.

Features:

- C/C++/Python support
- Widely used in AI and HPC
- Optimized for NVIDIA GPUs

Applications:

- Deep learning frameworks (TensorFlow, PyTorch)
- Scientific simulations

OpenCL (Open Computing Language)

Open standard supported by:

- NVIDIA
- AMD
- Intel

Advantages:

- Hardware independent
- Portable across platforms

CUDA vs OpenCL

Feature	CUDA	OpenCL
Developer	NVIDIA	Open standard
Hardware support	NVIDIA GPUs only	Multiple vendors
Performance	Higher on NVIDIA	Portable
Ease of use	Easier	More complex

4.9 GPU in Machine Learning and Big Data

GPUs revolutionized **AI and Big Data analytics**.

Applications

1. Deep Learning

Neural networks require:

- Matrix multiplications
- Gradient calculations

GPUs speed up training dramatically.

2. Scientific Computing

Examples:

- Weather forecasting
- Molecular simulations
- Astrophysics

3. Big Data Analytics

Frameworks using GPUs:

- RAPIDS
- Spark GPU acceleration
- TensorFlow & PyTorch

4.10 GPU + Big Data Framework Integration

Modern Big Data tools support GPU acceleration.

Examples:

- Apache Spark + GPU acceleration
- Hadoop GPU clusters
- GPU-enabled cloud services

This combination enables:

- Real-time analytics
- Faster machine learning
- High-throughput processing

4.11 Challenges of Using GPUs

Despite advantages, GPUs present challenges:

- Requires specialized programming
- Data transfer between CPU and GPU can be slow
- Expensive hardware
- Memory limitations

Therefore, GPUs are used alongside CPUs in **heterogeneous computing systems**.

4.12 Heterogeneous Computing

Modern HPC systems combine:

- CPUs → control and coordination
- GPUs → heavy computation

This hybrid model provides **maximum performance**.

SUMMARY

Key points:

- GPUs provide massive parallel processing.
- Big Data analytics relies heavily on GPUs.
- CUDA and OpenCL enable GPU programming.
- GPU + Cloud + Big Data frameworks = Modern HPC.

REVISION QUESTIONS

Short Answer

1. Define GPGPU.
2. Explain SIMD processing.
3. Differentiate CUDA and OpenCL.
4. Why are GPUs better for machine learning?
5. What is heterogeneous computing?

Essay Questions

1. Compare CPU and GPU architectures and their roles in Big Data analytics.
2. Explain how GPUs accelerate machine learning and Big Data processing.
3. Discuss the challenges of using GPUs in HPC systems.

5. Introduction to Apache Spark

Apache Spark is a fast, distributed data processing engine designed for **big data analytics and machine learning**. It was created to overcome the speed and usability limitations of Hadoop MapReduce by enabling **in-memory processing** and a **unified analytics engine**.

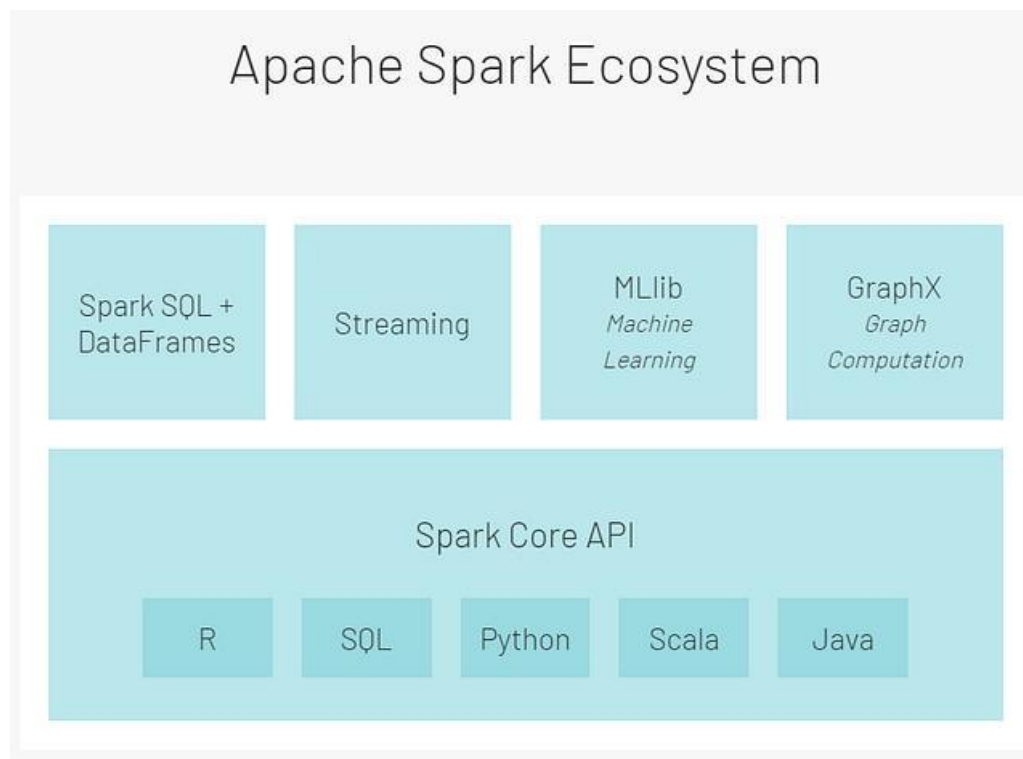
Spark was originally developed at UC Berkeley's AMPLab and is now maintained by the **Apache Software Foundation**.

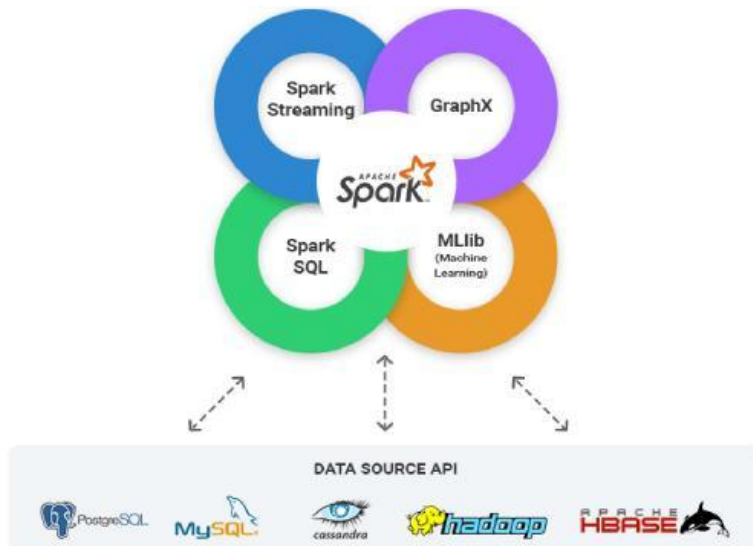
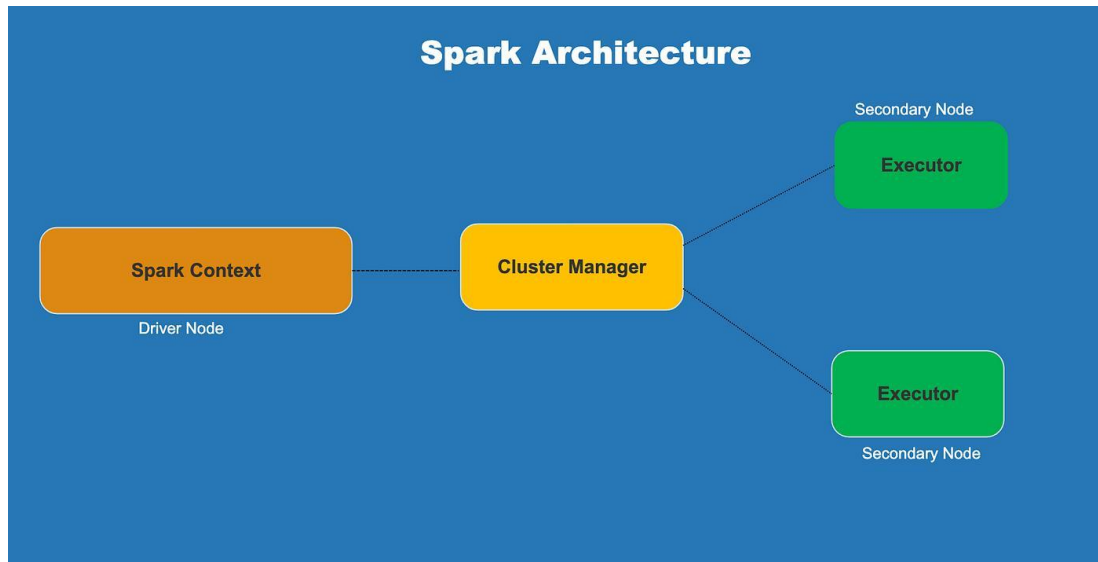
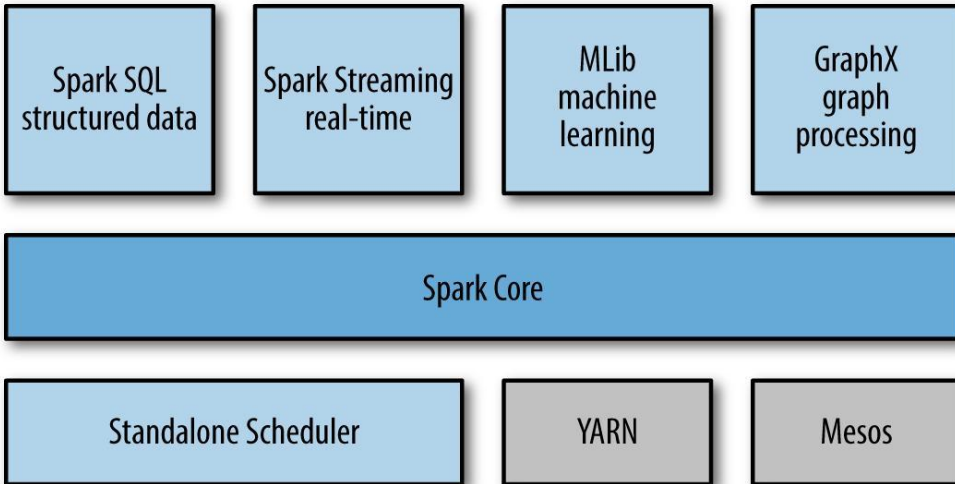
Why Spark became popular

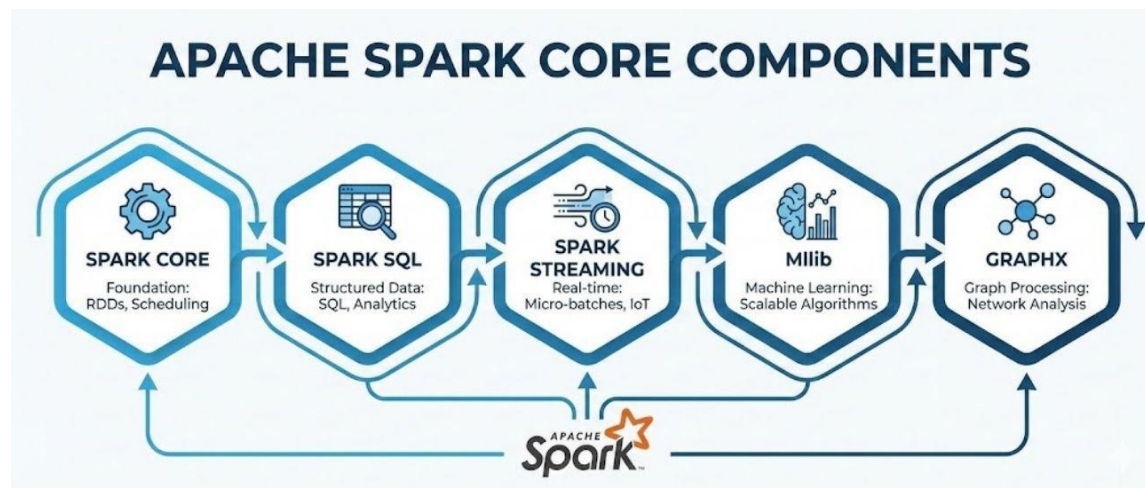
- Processes massive datasets **100× faster** than MapReduce (in memory)
- Supports multiple languages: **Python, Java, Scala, R**
- Handles **batch, streaming, ML, SQL, graph processing** in one platform
- Easy integration with Hadoop ecosystem

5.1 Apache Spark Ecosystem Overview

Spark is not just a single tool — it is a **complete analytics ecosystem**.





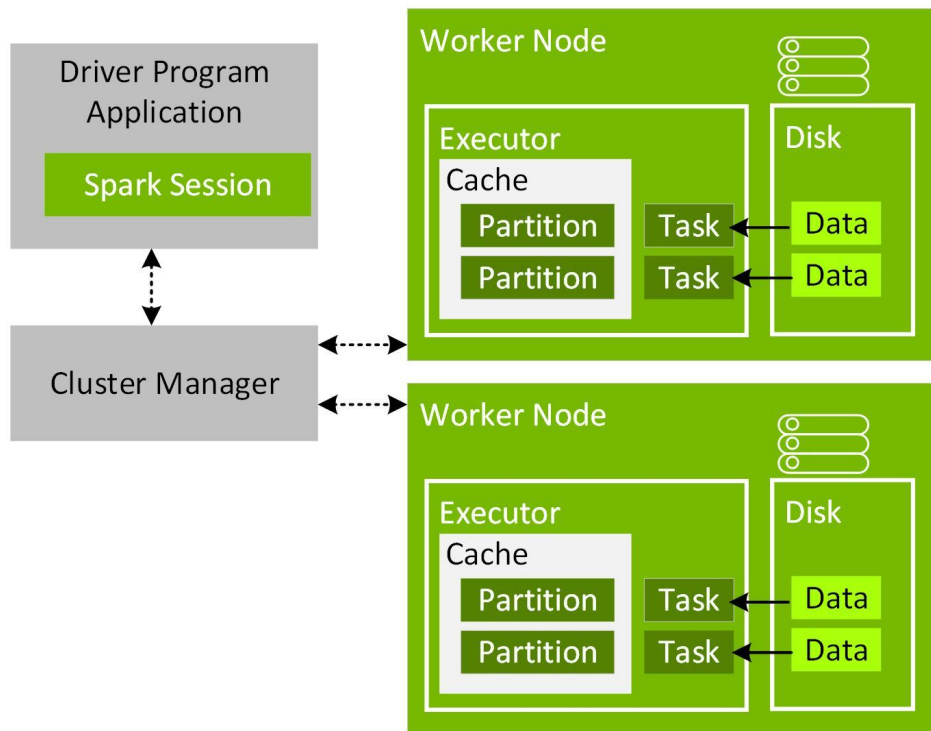
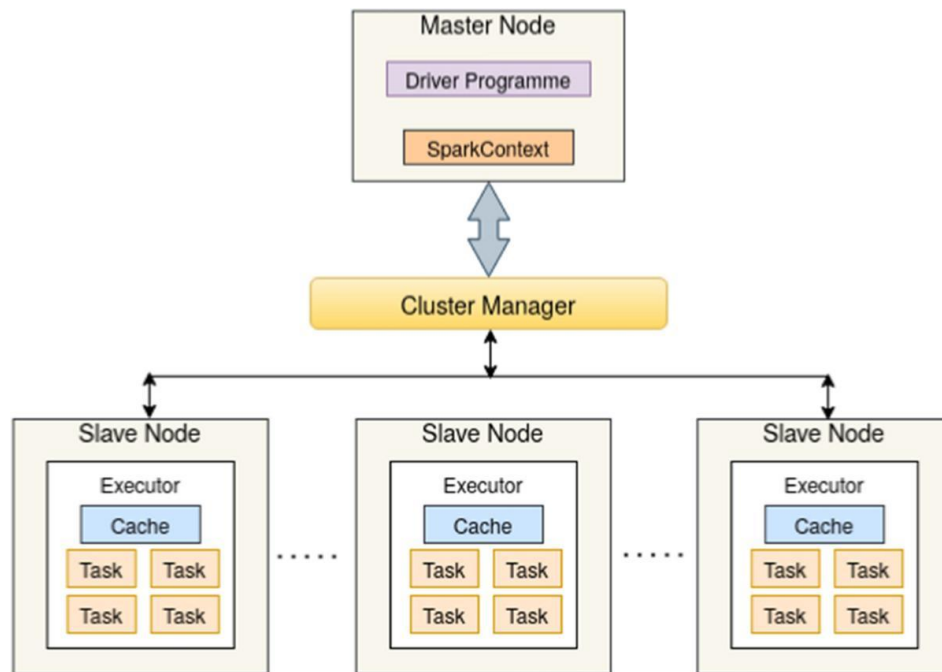


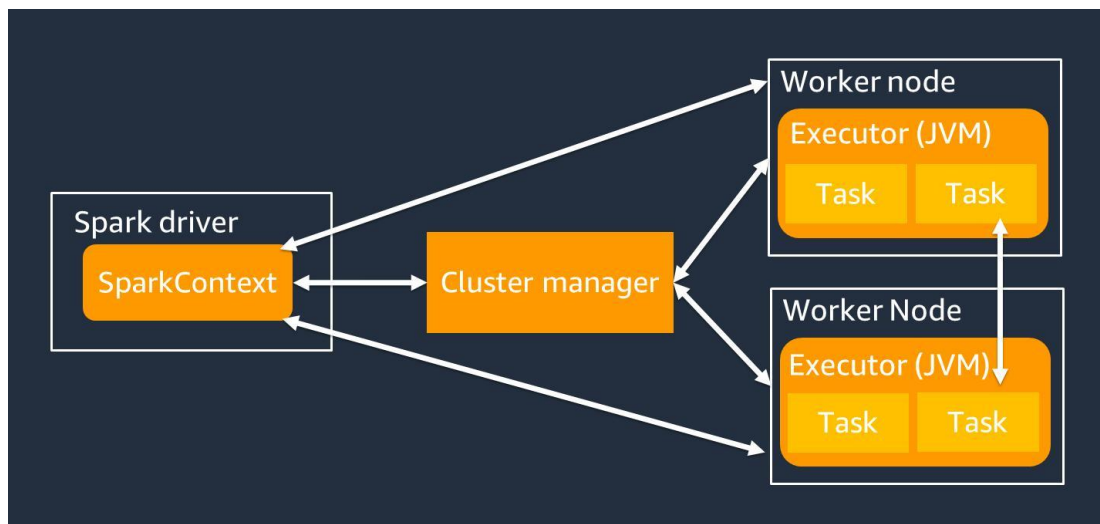
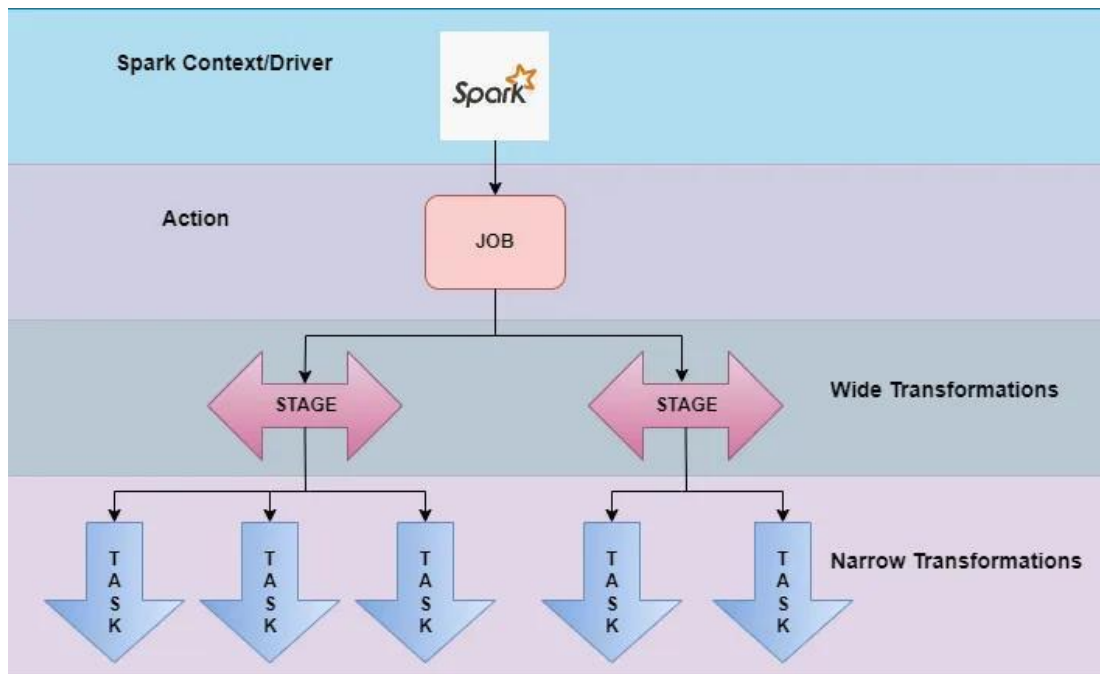
Core Components

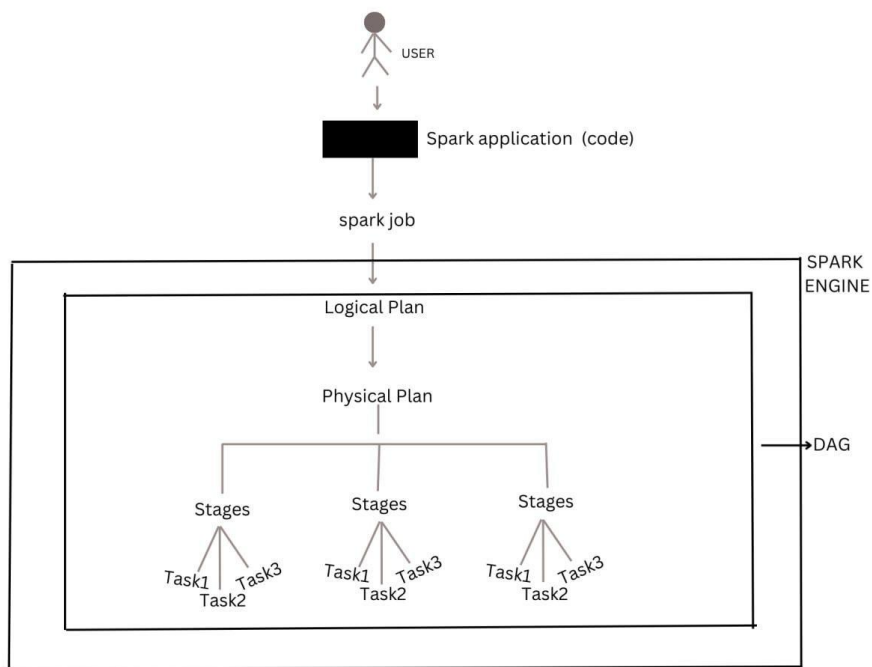
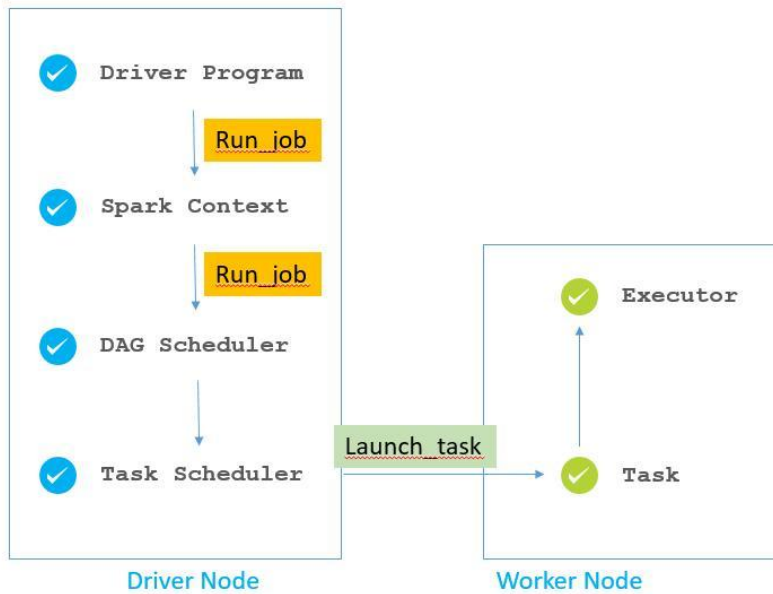
Component	Purpose
Spark Core	Engine for distributed processing
Spark SQL	Structured data & SQL queries
Spark Streaming	Real-time stream processing
MLlib	Machine learning library
GraphX	Graph processing engine

5.2 Apache Spark Architecture

Spark uses a **Master–Worker distributed architecture**.







5.2.1 Main Architecture Components

1 Driver Program

The **Driver** is the brain of Spark.

Responsibilities:

- Contains the **main application**
- Creates **SparkContext**
- Breaks tasks into smaller units
- Sends tasks to executors
- Collects results

Think of the Driver as a **project manager**.

2 Spark Context

The **SparkContext** connects the driver to the cluster.

Functions:

- Allocates resources
- Creates RDDs/DataFrames
- Communicates with Cluster Manager

It is the **entry point** to Spark.

3 Cluster Manager

The cluster manager allocates resources across nodes.

Supported managers:

- Standalone cluster manager
- Hadoop YARN
- Kubernetes
- Apache Mesos

4 Worker Nodes

Worker nodes are machines that perform the actual work.

Each worker runs:

- Executors
- Tasks

5 Executors

Executors are processes that run tasks.

Responsibilities:

- Execute computations
- Store data in memory
- Return results to driver

Executors are the **workers doing the real job**.

6 Tasks

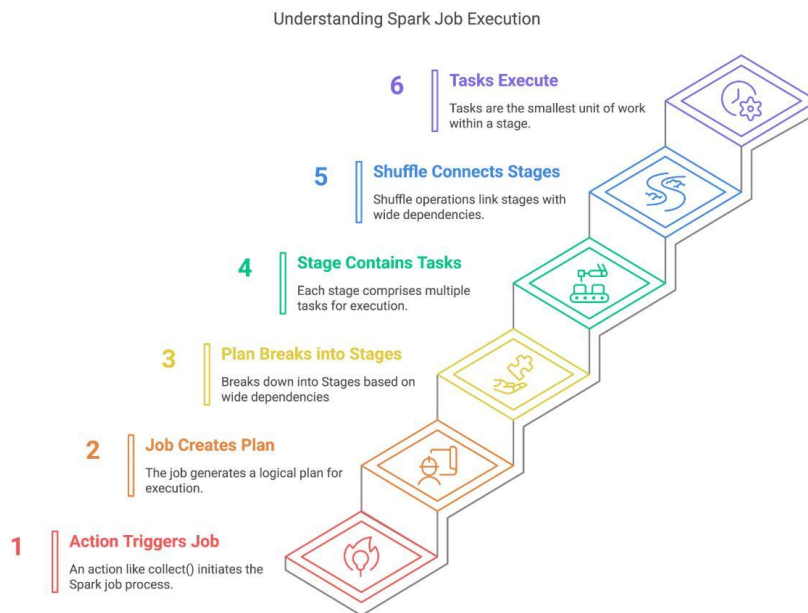
A **task** is the smallest unit of work.

Spark job flow:

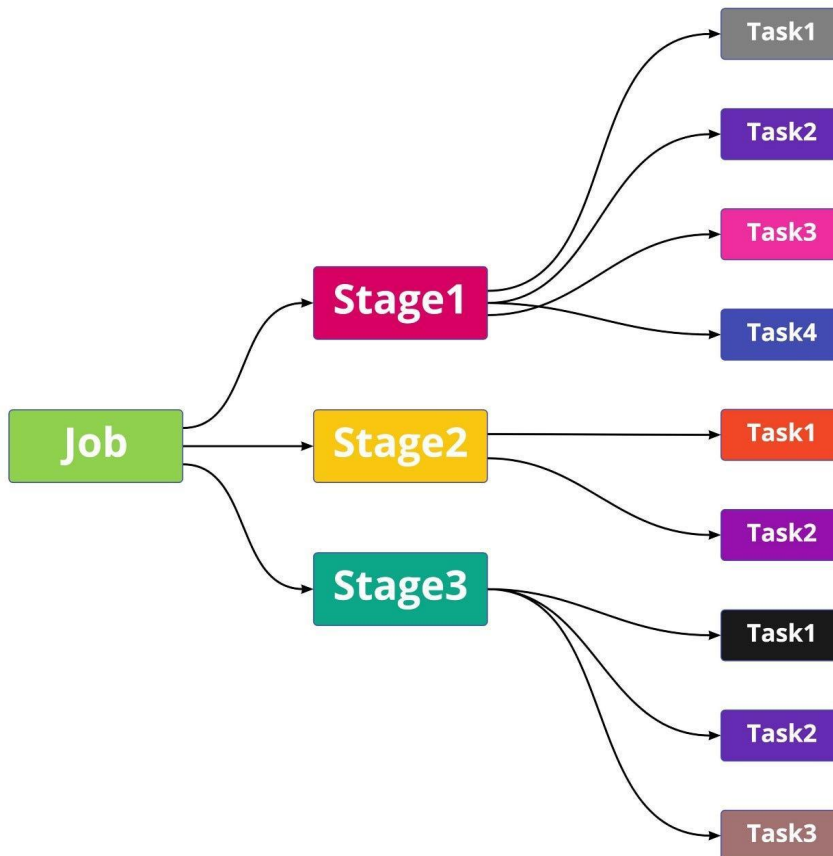
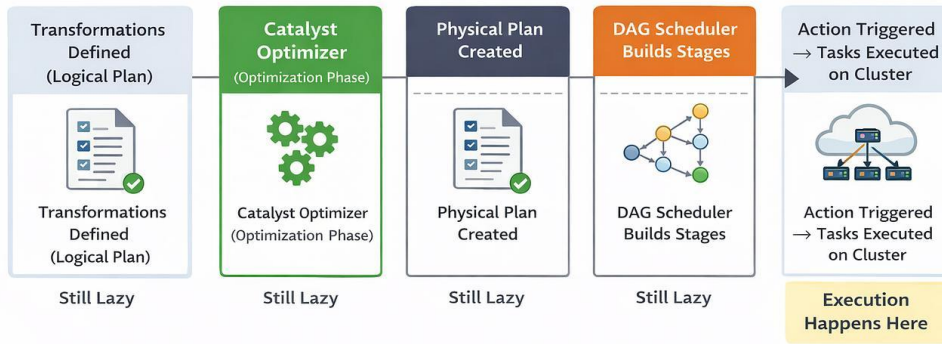
1. User runs program
2. Driver splits job into stages
3. Stages split into tasks
4. Tasks run on executors

5.3 Spark Execution Model

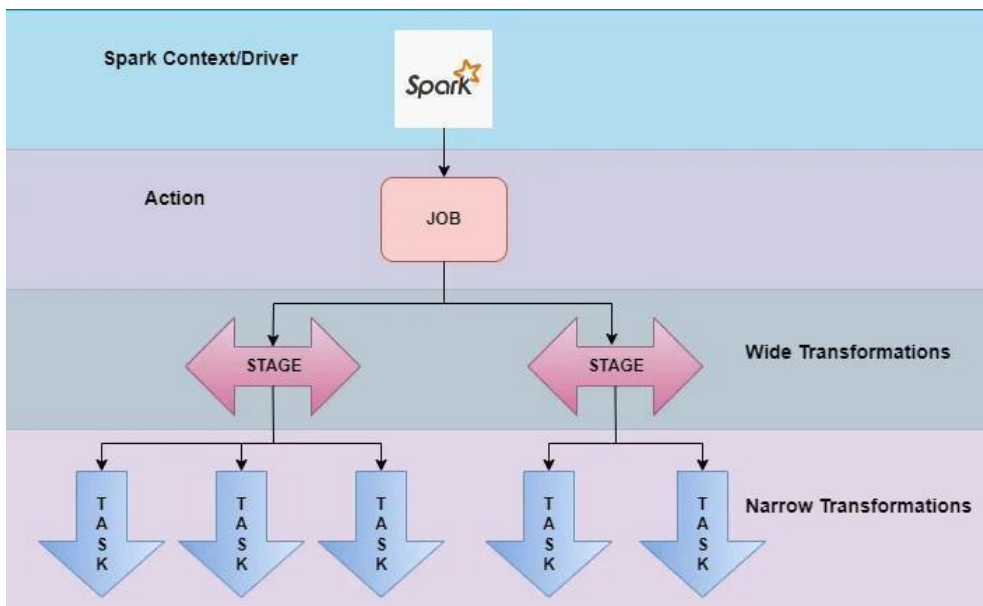
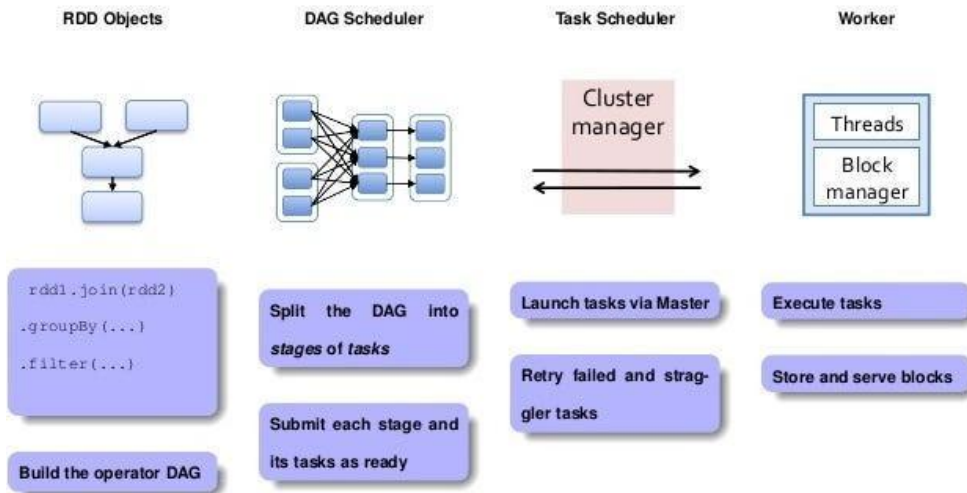
Step-by-step job execution

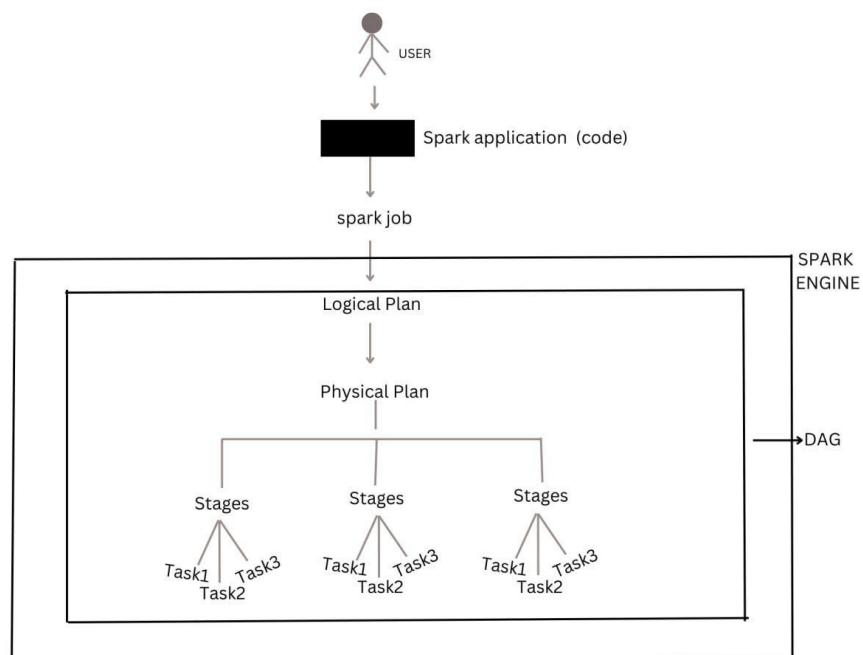
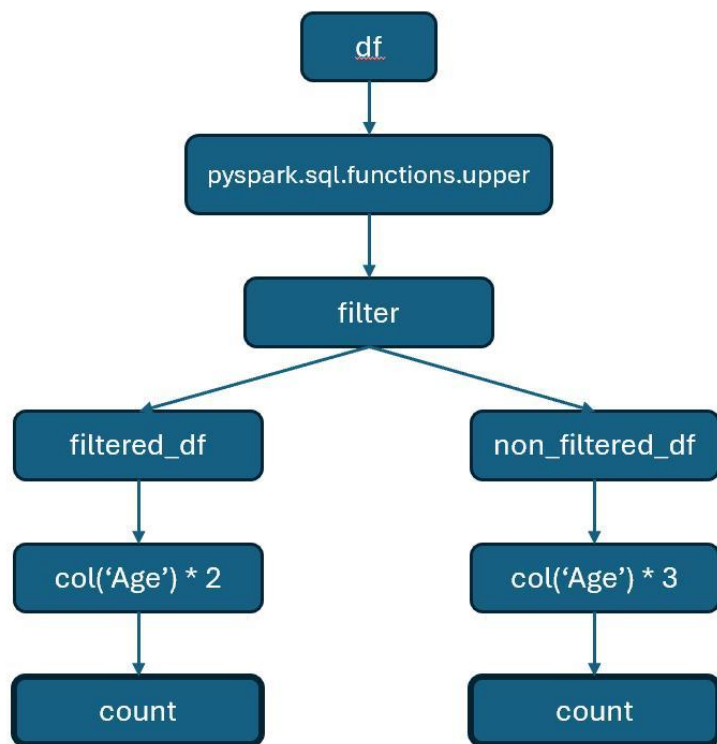


Spark's Lazy Evaluation Workflow



Lifetime of a Job in Spark





Step 1 — User Application

User writes code in Python/Scala/Java.

Example:

```
data = spark.read.csv("sales.csv")  
  
data.filter(data.amount > 1000).count()
```

Step 2 — DAG Creation (Directed Acyclic Graph)

Spark converts operations into a **DAG** of tasks.

DAG = logical plan of computation.

Step 3 — Lazy Evaluation

Spark does NOT run immediately.

Execution starts only when:

- collect()
- count()
- save()
- show()

This allows Spark to **optimize the workflow**.

Step 4 — Job → Stages → Tasks

Spark divides work into:

Level	Description
Job	Entire application
Stage	Group of tasks
Task	Smallest execution unit

5.4 Spark Core Concepts

5.4.1 Resilient Distributed Dataset (RDD)

RDD = **fundamental data structure in Spark**

Properties:

- Distributed across cluster
- Fault tolerant
- Immutable
- Processed in parallel

RDD operations:

- Transformations (lazy)
- Actions (execute)

Transformations vs Actions

Transformations	Actions
map()	collect()
filter()	count()
flatMap()	saveAsTextFile()

5.4.2 DataFrames & Datasets

Modern Spark mainly uses **DataFrames**.

Advantages:

- Schema support
- SQL queries
- Better optimization
- Faster than RDD

Example:

```
df.select("name").filter(df.age > 20)
```

5.5 Spark Memory Management

Spark uses **in-memory computing**.

Traditional Hadoop:

- Writes intermediate results to disk ✗

Spark:

- Stores intermediate data in RAM ✓

This makes Spark:

- Much faster
- Ideal for iterative algorithms
- Perfect for machine learning

5.6 Fault Tolerance in Spark

Spark is fault tolerant via **Lineage**.

If data is lost:

- Spark recomputes it from original transformations.

Example:

RDD created from file → filter → map

If node crashes → Spark rebuilds lost partition automatically.

5.7 Key Features of Apache Spark

1. Lightning Speed

- In-memory processing
- DAG optimization
- Parallel execution

Speed comparison:

- MapReduce → disk based
- Spark → memory based

2. Unified Analytics Engine

Single platform for:

- Batch processing
- Streaming
- Machine Learning
- SQL queries
- Graph analytics

3. Scalability

Runs on:

- Laptop

- Cluster
- Cloud

Can scale to **thousands of nodes**.

4. Multi-Language Support

Supports:

- Python (PySpark)
- Scala
- Java
- R

5. Advanced Analytics Libraries

Spark SQL

Run SQL queries on big data.

MLlib

Machine learning algorithms:

- Classification
- Regression
- Clustering
- Recommendation

Spark Streaming

Real-time analytics:

- Fraud detection
- IoT monitoring
- Social media analysis

6. Fault Tolerance

Automatic recovery from failures using lineage.

☛ 7. Cloud Friendly

Works perfectly with:

- AWS

- Azure
- Google Cloud
- Hadoop clusters

5.8 Spark vs Hadoop MapReduce

Feature	Spark	Hadoop MapReduce
Speed	Very Fast	Slow
Memory Use	In-memory	Disk based
Ease of Use	Easy APIs	Complex
Real-time	Yes	No
Machine Learning	Built-in	External tools

5.9 Real-World Use Cases

Companies using Spark:

- Netflix → recommendations
- Uber → real-time analytics
- Facebook → large-scale processing
- Amazon → customer analytics

Final Summary

Apache Spark is:

- A **fast distributed engine**
- Uses **in-memory processing**
- Provides **unified analytics**
- Highly **scalable and fault tolerant**
- Ideal for **Big Data, AI, and real-time analytics**